



Today's agenda

↳ Inheritance

↳ Constructor chaining

↳ Polymorphism

↳ method overloading

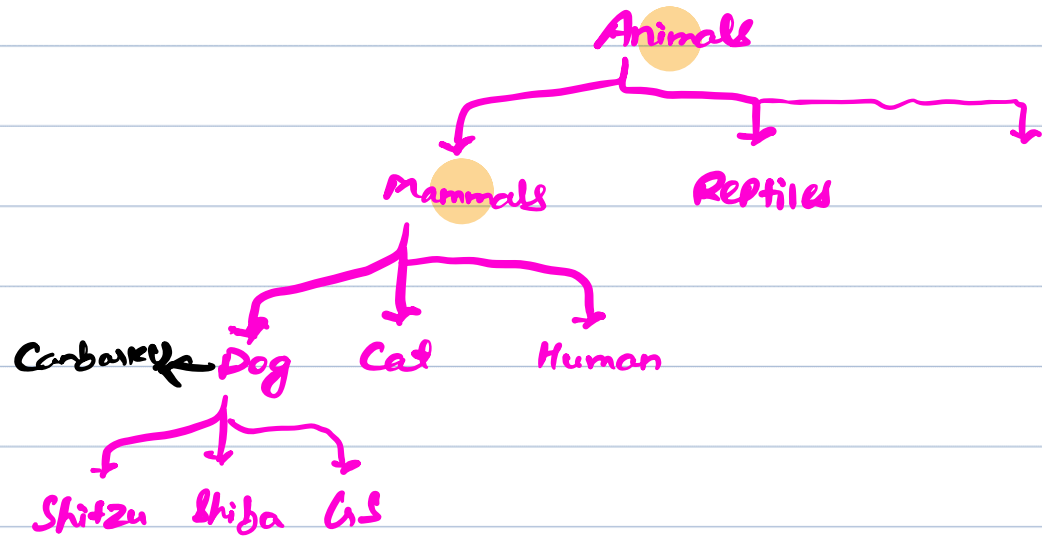


AlgoPrep



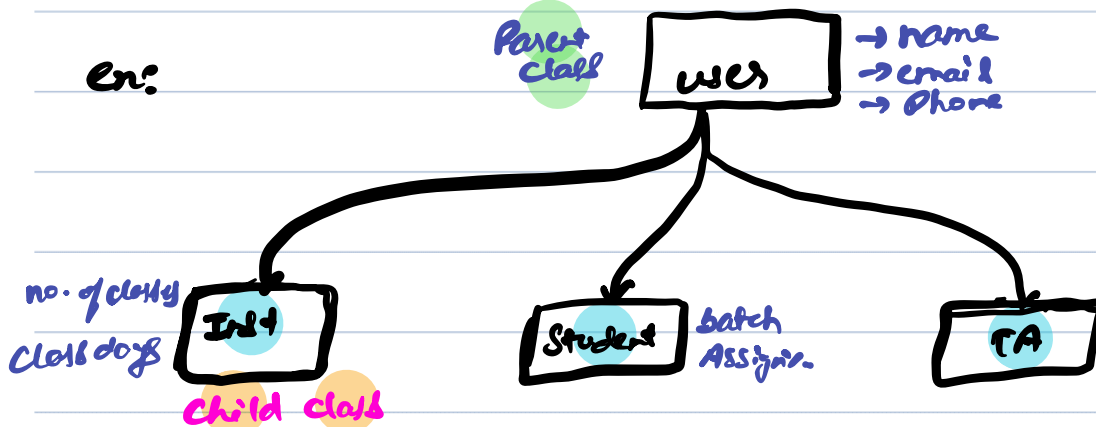
each class $\rightarrow$ entity

// Inheritance



$\rightarrow$ all the child class will have features of their parents but not vice-versa.

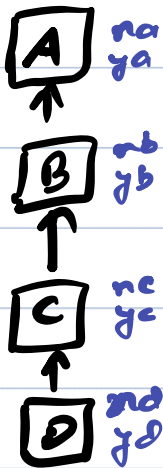
$\hookrightarrow$ Inheritance is like assigning hierarchy to class.



$\hookrightarrow$ Parent - Child Relation

$\hookrightarrow$ "extends" $\rightarrow$ keyword to inherit Parent class.

Constructor chaining



D obj = new D();

↑
triggers default constructor

D() → C() → B() → A()
initialize D attr. ← initialize C attr. ← initialize B attr. ← initialize A attr.

```
1 package Student;
2
3 public class A {
4     int xa;
5     int ya;
6
7     A(){ |
8         System.out.println("Constructor of A");
9     }
10 }
11
```

```
1 package Student;
2
3 public class B extends A {
4     int xb;
5     int yb;
6
7     B(){
8         System.out.println("Constructor of B");
9     }
10 }
11
```



```
A.java *B.java C.java X D.java *client.java
1 package Student;
2
3 public class C extends B {
4     int xc;
5     int yc;
6
7     C(int v1){
8         System.out.println("Constructor of C");
9     }
10 }
11
```

```
A.java *B.java C.java D.java X *client.java
1 package Student;
2
3 public class D extends C {
4     int cd;
5     int yd;
6
7     D(){
8         super(10);
9         System.out.println("Constructor of D");
10    }
11 }
12
```

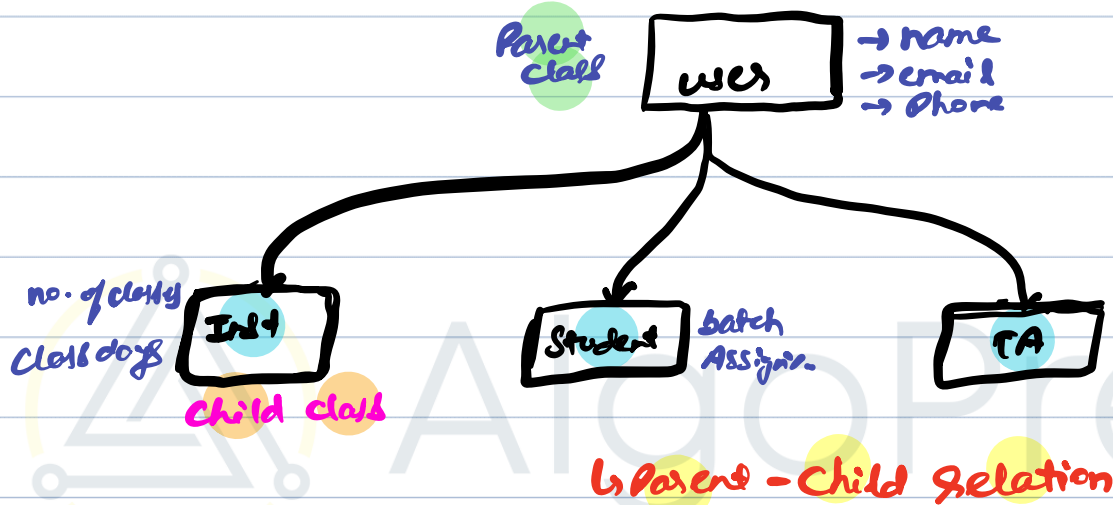
```
A.java *B.java C.java D.java *client.java X
1 package Student;
2
3 public class client {
4
5     public static void main(String[] args) {
6         //constructor chaining
7         D obj = new D();
8
9
10
11     }
12 }
13 }
14
```



Polymorphism

many forms

- How $\Rightarrow$
- ① Inheritance
 - ② method overloading
 - ③ method overriding



int i = 10

↳ user u = new user();

↳ student s = new student();

→ user u = new student(); → Possible

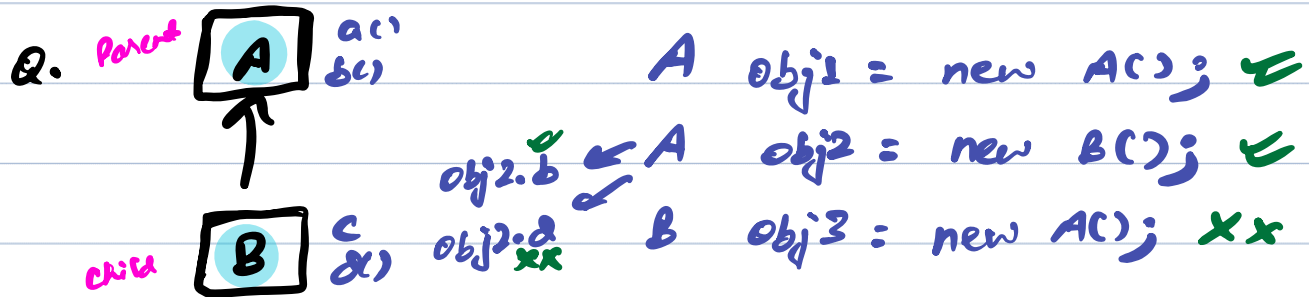
↳ u.email

↳ u.batch?? → Can't be accessed

```
user u = new user();
        new student();
        new instructor();
        new TA();
```



→ Student s = new user();
↳ Not Possible



AlgoPrep



→ compile time Polymorphism.

* Method Overloading

↳ A class having multiple methods with same name (with diff set of Param).

```
class Student {  
    void hello ( ) {  
        S.o.p ("Hello!");  
    }  
  
    void hello (int i ) {  
        S.o.p ("Hello2");  
    }  
}
```



Q)

```
class student {
```

```
    void hello (int j) {  
        s.o.p ("Hello!");  
    }
```

XX

```
    void hello (int i) {  
        s.o.p ("Hello2");  
    }
```

```
}
```

↳ names won't be considered in method overloading

Q)

```
class student {
```

```
    void hello () {  
        s.o.p ("Hello!");  
    }
```

✓✓

```
    int hello (int i) {  
        s.o.p ("Hello2");  
        return 0;  
    }
```

```
}
```



Q

```
class Student {
```

```
    void hello ( ) {  
        s.o.p ("Hello!");  
    }
```

XX

```
    int hello ( ) {  
        s.o.p ("Hello2");  
        return 0;  
    }
```

```
}
```

↳ return type won't be considered during method overloading.